BEVerage: Learning Navigable Bird's-Eye-View Representations from a Single Camera for Indoor Local Planning

Mid Submission

Anay Rungta Advaith Sudarshan Siddharth Sinha

November 5, 2026

1 Problem Statement

Setting. A cylindrical ground robot (radius $r = 0.2\,\text{m}$, height $1.0\,\text{m}$) moves through a static indoor scene with discrete actions (forward $0.25\,\text{m}$, turn $10°$). Its only sensor at test time is one forward-facing RGB camera at height $h = 0.88\,\text{m}$ with a $90°$ horizontal field of view and no pitch. At time t the robot observes an image I_t and is given a point goal g on the same floor (relative position from odometry).

Local map. The robot plans on an egocentric bird's-eye-view (BEV) grid $\mathcal{G}$ of 80×80 cells at $5\,\text{cm}$, covering $4\,\text{m}$ ahead and $\pm 2\,\text{m}$ sideways. A *representation* $\mathcal{R}$ assigns each cell $c \in \mathcal{G}$ a value $\mathcal{R}(c)$: Each representation comes with a fixed, untrained planner $\pi_{\mathcal{R}}$ that turns the map (and the goal) into an action.

Problem. Given a family of candidate representations $\{\text{R1}, \ldots, \text{R7}\}$ (Section 2), we want

(i) the representation $\mathcal{R}^\star$ whose planner navigates best when its map is imperfect: high success and SPL on ground-truth maps, and the slowest degradation as the maps are corrupted the way a network corrupts them;

(ii) a predictor $f_\theta : I_t\,(\,, g) \mapsto \hat{\mathcal{R}}^\star_t$ trained on simulator ground truth that predicts $\mathcal{R}^\star$ from a single image, *including* cells that are occluded or lie in the camera's blind zone (the first $h/\tan(\text{VFOV}/2) \approx 1.17\,\text{m}$ in front of the robot, where the floor is never in the image);

(iii) an evaluation that measures map error and closed-loop navigation together, so that the loss caused by perception (oracle gap) is separated from the loss caused by the representation or the planner.

Scope and assumptions. Simulation only (Habitat on HM3D), wherein we have static scenes, single-floor episodes. Privileged depth, pose, semantics and navmesh are used only to build labels and references, never as network input at test time. An episode succeeds if the robot ends within $0.2\,\text{m}$ of the goal within 500 steps.

2 Progress: Candidate Representations

Stage 1 is complete: one pipeline turns privileged Habitat data (depth, pose, semantics and a robot-sized navmesh) into aligned per-frame labels for seven candidate representations. For each we describe how it is built and where it falls short, both as a label and as a *learning target*: what a network must infer beyond the image, what kind of loss it implies, and whether that loss rewards what the planner needs.

2.1 Shared pipeline

Setup. The camera (480×640, $90°$ HFOV, $f = 320\,\text{px}$) sits $h = 0.88\,\text{m}$ above the floor with no pitch. The egocentric grid has 80×80 cells of $s = 5\,\text{cm}$, covering $4\,\text{m}$ ahead and $\pm 2\,\text{m}$ sideways.

Back-projection and height bands. A pixel (u, v) with depth D becomes the agent-frame point which is then binned into the grid. Its height y assigns a band: *floor* $[-0.15, 0.10)\,\text{m}$, *body* $[0.10, 1.0)\,\text{m}$ or *overhead* $[1.0, 2.0)\,\text{m}$ and other points are dropped. Counting points per band and cell gives $N \in \mathbb{N}^{3\times 80\times 80}$.

Visible and amodal labels. *Visible* labels use only the current depth image, so occluded cells, cells outside the field of view and the blind zone are unknown. *Amodal* labels describe the true layout: for each floor we fuse ≈2,000–3,000 views (navigable points on a $0.5\,\text{m}$ grid, eight headings each) into a world-aligned map, with heights measured relative to each viewing agent's floor. The per-frame label is the $4\,\text{m}$ egocentric crop of this map, by nearest-cell lookup of the rotated cell centres. R4, R6 and R7 are computed on the whole-floor navmesh raster and are amodal by construction. Visible and amodal R1 agree on ≈96% of commonly observed cells, and the amodal labels fill 83–90% of the blind zone.

Shared assumptions. (A1) Depth, pose, semantics and navmesh are exact and scenes are static. (A2) The camera is level and floors are locally flat; on ramps, points move between bands. (A3) Bands and navmesh are tied to one robot body; changing it means regenerating every label. (A4) The fused map is treated as the truth, and amodal "unknown" is read as "inside something solid". (A5) Episodes stay on one floor. (A6) Nearest-cell resampling shifts boundaries by up to $2.5\,\text{cm}$ when the robot is not axis-aligned.

2.2 R1 Occupancy and R2 Height-sliced occupancy

R2 is per-band presence, with number of points $n_{\min} = 2$ for a single frame and 5 for the fused map. R1 collapses it for a planner. HM3D meshes contain floating scan

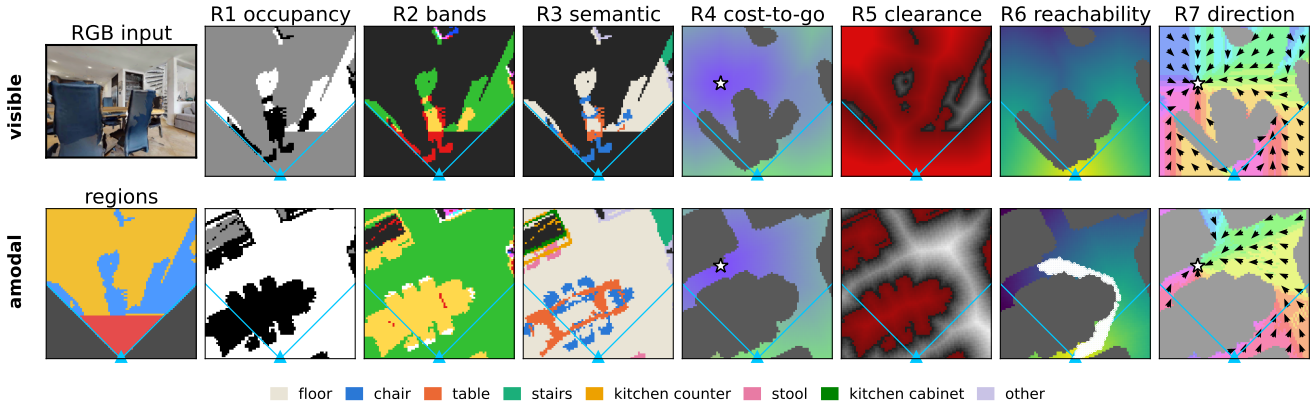


Figure 1: All seven representations for one frame of HM3D scene 00808, from the current depth image only (*visible*, top) and from the fused floor map (*amodal*, bottom). The robot is at the bottom centre and cyan lines bound the 90° field of view. Regions: blue seen, yellow occluded, red blind zone, grey out of view. R1: white free, black occupied, grey unknown. R2: green floor, red body, blue overhead (yellow = floor and body, e.g. under the table). R3: most common classes, dark = unknown. R4: blue toward the goal, green near zero, grey unreachable (colour scale per panel). R5: red inside obstacles, white ≥ 1 m clear. R6: yellow near, purple far; white dots mark a shortest robot–goal path ($r_6 + r_4 = 0$). R7: hue and arrows give the direction. Star: the route subgoal in the crop (goal 4.6 m away). Visible R4/R7 are computed to that subgoal on the visible R1 costmap, and visible R5/R6 are derived from visible R1, as in the closed loop (Section 4). Visible R5 is almost all obstacle because unknown cells count as obstacles.

fragments that, after fusion, can push a doorway cell past $n_{\min}$. The fused map therefore also requires $N_{\text{body}} \geq 0.05\,(N_{\text{floor}} + N_{\text{body}})$: fragments have a 0.5–1% body share, real obstacle edges 25% or more.

Limitations and learnability. R1 is binary, so one wrong cell closes a doorway; flipping 2% of cells at random takes oracle success from 0.96 to 0.00 (Figure 3). *Loss and planner disagree:* Per-cell cross-entropy weighs every cell equally, while navigation depends on a few doorway and blind-zone cells, and free cells dominate the class balance. A network can reach high accuracy and still close the one doorway that matters. Its Mostly inferred: a visible-only label covers only ≈30% of the cells the amodal label knows, so roughly 70% of the supervision is occluded or blind-zone space the network must hallucinate. Depth precision: for floor points, a depth-scale error ϵ shifts the height by ϵh (4.4 cm at 5%), almost half the 10 cm floor/body margin, and the position by ϵD (15 cm at 3 m), enough to close a doorway. *Inconsistent targets:* visible and amodal labels use different $n_{\min}$ and only the amodal one applies the 5% filter. That filter mostly trims 5 cm off obstacle edges (96% of removed cells), so it shrinks obstacles, which is the unsafe direction.

2.3 R3 Semantic BEV

A per-scene registry maps the semantic sensor's instance ids to categories (140–220 per HM3D), unannotated pixels are discarded. ceiling points are discarded, $\text{R3}(c) = \arg\max_k N_k(c)$, or *unknown* below $n_{\min}$ points. The amodal version votes over the fused floor and is cropped. Visible and amodal R3 agree on ≈95% of cells. Its main use is defining object goals.

Limitations and learnability. Large surfaces win: floor points vote, so a cell holding a chair leg usually reads "floor", and thin objects disappear where they matter for collisions. *Long-tailed classes:* ids differ between scenes and most of the 140-220 categories cover a handful of cells, so training needs a shared vocabulary and class rebalancing, or a merge into a few goal classes. *Amodal semantics is mostly a guess:* the image constrains free space behind a wall far better than *which object* is behind it, so amodal R3 supervision outside the view is close to label noise. *Gaps:* unannotated mesh becomes unknown, and objects above 1 m are excluded.

2.4 R4 Goal-conditioned cost-to-go

One 8-connected Dijkstra sweep from the goal over the whole-floor navigable mask gives the geodesic distance $d(c, g)$ for every cell. Because the distance is symmetric, the sweep serves every robot pose. The label is relative to the robot's cell c_0, $r_4(c) = d(c, g) - d(c_0, g)$: 0 at the robot, negative toward the goal, $+\infty$ where unreachable. The subtraction makes it plausible to learn, since the absolute distance to a distant goal cannot be seen but differences inside the window are bounded. The planner descends the slope (greedy, or A* with an uphill penalty). Goals are sampled 2-12 m away independently of the heading, so goals behind the robot are included.

Limitations and learnability. *Entangles map, goal and planning:* a network must learn the layout, use the goal as an input and imitate a shortest-path solver all at once, with a forward pass per goal. With one goal per frame in the data it can memorise rather than condition on the goal. The alternative is to predict a goal-free map and *compute* r_4 in milliseconds. *Non-local:* which doorway leads to a distant goal is decided outside the image, so the slope near the robot is often not inferable. Then the errors would be structured (a confidently wrong doorway), unlike the independent per-cell noise of our oracle study. *Per-cell loss, global requirement:* greedy descent needs a field without spurious local minima, but an L1 loss does not penalise them, a small dent costs little loss and traps the planner. Unreachable cells ($+\infty$) also need a separate mask head. *Label artefacts:* 8-connected paths overestimate length by up to 8.2%, can disconnect passages the navmesh keeps (then

$r_4 = \infty - \infty$, unguarded), and follow the navmesh while R1/R5 follow fused depth.

2.5 R5 Clearance field

With obstacles $\mathcal{O} = \{c : \mathrm{R1}(c) \neq \text{free}\}$, R5 is the signed Euclidean distance to $\mathcal{O}$ (positive outside, negative inside), computed with exact distance transforms on the whole floor, cropped, and clipped to ±2 m. Computing it before cropping keeps edge cells correct when an obstacle lies just outside the window. A planner turns it into an inflation cost: ∞ below $0.75\,r$, else $1 + 4e^{-(r_5 - r)/0.2}$ per metre (5× at one robot radius, ≈ 1× in the open). A small error thus makes a doorway *more expensive* rather than *closed*: with obstacles grown by 15 cm, R5 keeps 0.93 oracle success where the other planners fall to 0.39-0.54 (Figure 3). Its sign matches amodal R1 on 100% of cells.

Limitations and learnability. R5 is the easiest field to learn: bounded, smooth, local, and a deterministic function of R1, so a network can predict it directly or predict R1 and compute it exactly. *Loss weighting:* the planner only cares about values near r (0.15-0.4 m), where the cost changes fastest; a uniform L1 spends most of its effort on open space and wall interiors, so near-obstacle cells need extra weight. *Unknown is an obstacle:* correct for the fused map, but on a visible-only R1 the blind zone becomes an obstacle and the robot's own cell reads negative. *Edge error:* R5 derived from a predicted crop cannot see obstacles beyond it and overestimates clearance at the edges, the unsafe direction. *Goal-free:* it must be paired with a subgoal, R4 or R7.

2.6 R6 Egocentric reachability

R6 is the R4 sweep rooted at the robot, with nothing subtracted: $r_6(c) = d(c_0, c)$. A cell just behind a wall reads the detour through the door; on scene 00800, 17% of reachable cells exceed 1.5× their straight-line distance plus 0.2 m. Being goal-free, R6 plans with a heading cue (straight-line distance or R4).

Limitations and learnability. *Unbounded, non-local targets:* values reach 20 m or more in one window and are not clipped for training. The largest ones, behind walls, depend on where the doors are, which is usually outside the image. Those cells are effectively unpredictable, yet they dominate an L1/L2 loss. Clipping, a log scale, or supervising only the reachability mask beyond a few metres would fix this. Near the robot the field is local and learnable. *Robot-rooted:* it changes every step, and with the straight-line cue it heads into dead ends deeper than the window. *Optimistic oracle:* the oracle uses a 4 m disc around the robot, while the label is a forward crop.

2.7 R7 Direction field

From the same $d(\cdot, g)$, each cell points to its lowest 8-neighbour (the next cell of a shortest path). Following the pointers six steps (0.3 m) gives a point down the path; the direction to it, minus the robot's yaw, is quantised into 16 bins of 22.5° (bin 0 ahead), plus a "no direction" class. The lookahead is meant to give a continuous angle rather than eight grid directions. R7 is a bounded classification target, planned on by following the arrows or by A* with a misalignment cost.

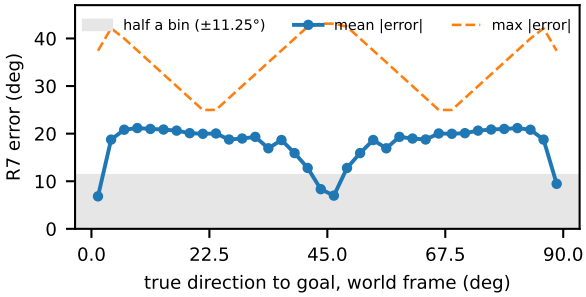


Figure 2: R7 heading error on an empty floor (cells > 1 m from the goal) against the true world-frame direction. Away from the grid axes and diagonals, arrows snap toward multiples of 45°: mean 17°, max 43°.

Limitations and learnability. *Snapping to 45° (measured):* many staircase paths tie for shortest on an 8-connected grid, and the lookahead follows the first leg of one of them, so labels are off by 17° on average and up to 43° (Figure 2; the stripes in Figure **??**). The error follows the *world* grid while the label is egocentric, so it depends on the robot's world heading, which the image cannot show, i.e label noise a network cannot explain. Grid planners are unaffected (oracle success 1.00, SPL 0.98), which is why the oracle missed it. Fast marching or any-angle paths would remove it, together with the length bias in R4/R6. *Circular classes:* plain cross-entropy treats a one-bin miss like a 180° miss, the loss needs circular label smoothing or an angular (e.g. von Mises) term. *Reversals:* where going left or right around an obstacle costs the same, arrows flip 180° between neighbours. The target is bimodal there, and convolutional networks blur such edges, averaging the two modes gives "straight ahead", into the obstacle. *No magnitude:* routes cannot be compared, so arrow-following cannot escape loops. *Wall-hugging:* shortest paths have minimal clearance and the lookahead cuts corners. *Quantisation* adds ±11.25°, and R7 shares R4's goal dependence and non-locality.

2.8 Summary

The goal-free, bounded maps (R1, R2, R5) are the most natural *prediction* targets: they need only what the image shows plus amodal completion, and any goal-conditioned field can be computed from them exactly. The goal-conditioned fields (R4, R7) are the most robust *planning* inputs under noise, but their values depend on geometry far outside the image. This tension decides what Stage 3 predicts. The noise study (Section 4) sharpens it: R5 is the most tolerant of geometric error, and the goal-conditioned fields are robust only when a search-based planner (A*) consumes them.

3 Progress: Evaluation Metrics

3.1 Reference maps

- **Simulated LiDAR.** Four 90° depth cameras at 0.3 m give a 360° scan. In planar mode, obstacles are marked where rays hit and free space is carved along each ray. In 3D mode (16 rings, ±15°), body-band hits are obstacles, so table tops are seen. The LiDAR was chosen as reference because that was asked for as one of the baselines in the MR website.
- **Amodal labels** (Section 2) for every representation, including occluded cells and the blind zone.
- **Visible labels**, for what is measurable from the current frame alone.

3.2 Map quality

Counts are summed over frames before ratios are taken, overall and per group: distance from the camera (0–1, 1–2, 2–3, 3–4 m) and region (visible, occluded, blind zone, outside the field of view).

- **R1:** IoU of free and of occupied cells and their mean (mIoU), free-space precision (the safety-relevant one), and coverage (share of reference-known cells the source labels at all).
- **R2:** per-band IoU. **R3:** mean class IoU.
- **R4–R6:** L1 error on cells both maps define, plus agreement on where they are defined (reachability agreement).
- **R7** (and R4's slope directions): bin agreement, within-one-bin agreement, mean angular error.
- **Free-space reliability range:** the distance at which free-space precision (in 0.25 m bins) first drops below 0.8.

3.3 Navigation

We evaluate under two settings: (a) the whole-floor oracle (plan once on the full floor map, under controlled noise), and (b) the egocentric closed loop (only the 4 m map at each step, subgoal = privileged global route projected onto the grid).

- **Success:** within 0.2 m of the goal in at most 500 steps.
- **SPL**: SPL $= \frac{1}{N}\sum_{i=1}^{N} S_i \frac{\ell_i}{\max(p_i,\ell_i)}$, where S_i is success, ℓ_i the shortest-path length and p_i the driven length.
- **Collisions** (Habitat's slide-along-the-wall events). Success alone hides them, because Habitat slides the robot along obstacles instead of stopping it. We report the share of episodes with at least one collision. The per-episode mean is dominated by the few episodes that slide along a wall until the step budget runs out.
- **Path length relative to the LiDAR-costmap planner** on episodes both complete.
- **Subgoal reach rate:** share of replans whose local path reaches the subgoal.
- **Oracle gap:** success of the same planner on ground-truth labels minus success on the evaluated source, on identical episodes. This isolates the cost of perception error.
- **Statistics:** 95% Wilson intervals on success rates; because all methods run on the same episodes, differences are tested with an exact McNemar test on the discordant episodes.

	Whole-floor oracle			Closed loop (4 m crop), 200 episodes				
	amodal			amodal			visible	vis.+fusion
Planner	Succ.	SPL	Coll.	Succ.	SPL	Coll.	Succ.	Succ.
navmesh	1.000	1.00	.57	–	–	–	–	–
R1	.974	.96	.49	.955	.94	.32	.720	.925
R2	.974	.96	.49	.955	.94	.32	.715	–[b]
R4 greedy	1.000	.98	.60	.995	.99	.37	.730	.940
R4 A*	1.000	.99	.62	.880	.87	.34	.620	.940
R5 A*	.976	.89	.01	.960	.95	.35	.000[a]	.000[a]
R5+R4	–	–	–	.950	.93	.37	.000[a]	.000[a]
R6	1.000	.97	.57	.995	.98	.44	.600	.920
R6+R4	1.000	.98	.56	.865	.85	.39	.605	.920
R7 A*	1.000	.98	.41	.995	.99	.41	.695	.940
R7 follow	–	–	–	1.000	.99	.39	.725	.935

Table 1: Navigation on privileged labels, amodal vs. visible-only (all closed-loop columns on the same 200 episodes). Coll.: share of episodes with at least one collision. Oracle: 500 episodes. Fusion: log-odds over time with exact odometry. [a]R5 counts unknown cells as obstacles, so on visible labels the blind zone covers the robot's own cell and no plan exists (this needs fixing:)). [b]Fusion currently fuses only R1 so R2 passes through unchanged.

3.4 Noise model for the oracle study

- **Source noise** (fair across representations): obstacles grown or shrunk by k cells (5 cm each) in the body band *and* the navigable mask, after which every representation is rebuilt from the same corrupted source.
- **Label noise** (per family): random R1 flips; additive $\mathcal{N}(0,\sigma)$ on the R4/R5/R6 fields; angular jitter or random replacement of R7 directions.

Independent per-cell noise is easier to tolerate than the structured errors a network makes. We are considering a more structured noise model (for example, removing or shifting whole doorways or obstacles), but need Krish's feedback on this :)

4 Progress: Preliminary Results

Setup. All numbers are on the 10 HM3D minival scenes (00800–00809), with goals 2–12 m (oracle) or 2–10 m (closed loop) away. The whole-floor oracle runs 50 episodes per scene (500 per planner), the noise sweep 8 per scene (80 per cell, so a 95% interval is about ±0.05 near 1.0 and ±0.1 near 0.5), and the closed loop 20 per scene (200 per method). Every closed-loop method runs on the *same* episodes, so we compare methods with an exact McNemar test on the discordant episodes. Success rates carry 95% Wilson intervals. Besides the amodal labels, every closed-loop planner is also run on *visible-only* labels built from the current depth image, alone and fused over time; this is what a network supervised on visible labels could at best provide. Visible R5/R6 are derived from visible R1, and visible R4/R7 are the cost-to-go to the route subgoal on the visible R1 costmap (unknown cells 3× cost).

Clean labels are not the bottleneck. On amodal ground-truth labels every representation reaches at least

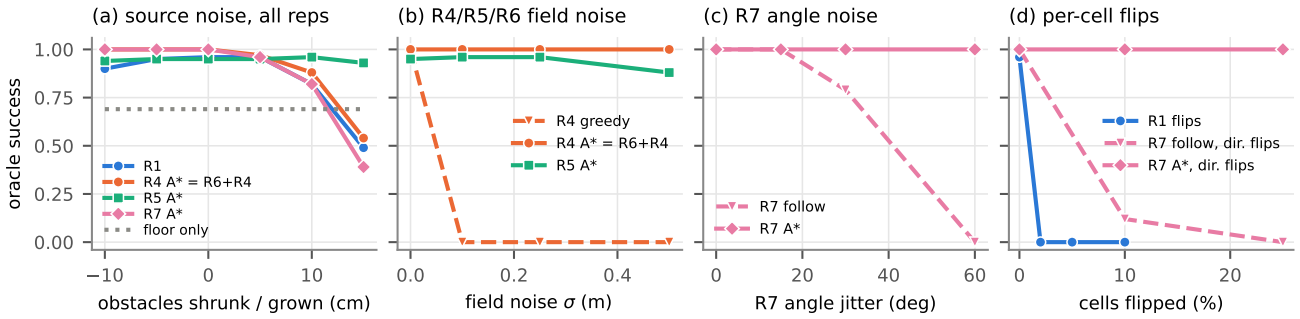


Figure 3: Oracle success under controlled noise (80 episodes per point, HM3D minival). (a) Obstacles shrunk or grown in the source map, after which every representation is rebuilt; R4 A* and R6+R4 coincide. (b) Gaussian noise on the scalar fields. (c) Angular jitter on R7. (d) Random flips of R1 cells or R7 directions. Solid: planners that search over the map (A*); dashed: planners that trust each cell (greedy descent, arrow-following).

0.97 oracle success (Table 1); the navmesh ceiling is 1.00. R1 and R2 give identical results because the planner reads the same body band. R5 trades SPL (0.89) for clearance: only 1.2% of its episodes touch an obstacle, against 41–62% for the others. On object goals (R3, four annotated scenes, 120 episodes) R1 reaches 0.83 against 1.00 for the navmesh.

Under noise, the planner matters as much as the representation. Figure 3 shows three patterns. (i) R5 is the most tolerant of *geometric* error: with obstacles grown by 15 cm it keeps 0.93 while every other planner falls to 0.39–0.54, because a narrowed doorway becomes expensive rather than closed. It is not immune to noise in its own values (0.88 at $\sigma = 0.5$ m). (ii) Binary occupancy is brittle: flipping 2% of R1 cells takes success from 0.96 to 0.00, because a few false obstacles close every passage. (iii) Smooth fields survive their own noise *only* with a planner that does not trust each cell: greedy descent on R4 drops to 0.00 at $\sigma = 0.1$ m and arrow-following on R7 to 0.12 at 10% flipped directions, while A* on the same fields keeps 1.00 throughout. Planning on the floor band alone reaches only 0.69 even on clean labels, since tables and shelves overhang visible floor, which supports keeping the body band (R1/R2). These are independent per-cell perturbations; a network's errors are structured (Section 2.4), so the A* results are an upper bound.

Oracle vs. closed loop. Most planners keep their oracle success when they see only the 4 m crop. The exceptions are R4 A* (0.88) and R6+R4 (0.87). This is a planner bug, not a property of the representation: the local planner targets the lowest-R4 cell in the crop, which is often behind a wall and disconnected from the robot within the crop. 22 of the 24 R4 A* failures are "no local plan", and greedy descent on the same map succeeds on all 24 of those episodes (0.995 overall). The whole-floor oracle never crops, so it cannot show this. Fused visible R4 A*, whose cost-to-go is computed inside the crop and whose minimum is therefore always reachable, scores 0.940, above the amodal version, which confirms that target selection, not R4, is at fault.

Visible-only labels: every representation loses. On single-frame visible labels every planner drops by 23–40 points against its amodal labels ($p < 10^{-10}$ for each; Table 1), to 0.60–0.73. They also fail the same way: almost every failure runs out of steps with a median of 0–3 forward moves, the robot turning in place (see the anomalies below). No representation escapes this, so the choice of target matters less than whether its labels cover what the camera cannot see. Fusing visible labels over time recovers most of the gap (0.92–0.94). Fused R1, R4 A* and R6+R4 are statistically indistinguishable from amodal ($p > 0.05$), while R4 greedy, R6 and both R7 planners remain 11–16 episodes worse ($p \leq 0.003$). R5 fails every visible episode because its obstacle set includes unknown cells. Treating unknown as free in a partial run (60 episodes, 4 scenes) gives 0.33 single-frame, below visible R1 on the same episodes (0.65), and 0.92 fused, matching amodal R5. R5 therefore depends more on amodal completion than R1, whose planner charges unknown cells 3× instead of forbidding or ignoring them.

Perception: what the sources lose. Single-frame maps from the camera's own depth (Table 2) know only the visible 30% of the cells the amodal label covers, and score 0 in occluded cells and in the blind zone. Visible-region mIoU is 0.91, so the loss is coverage, not accuracy. Navigation drops from 0.955 (amodal labels) to 0.72, a 23.5-point oracle gap. LiDAR covers the blind zone (blind-zone free IoU 0.92) and stays within 1.5 points of the labels. Accumulating exact depth with odometry recovers most of the gap: 0.925, with 44 episodes gained and 3 lost ($p < 10^{-9}$). Together with the visible-label results above, this is the strongest evidence so far that the predictor must be amodal, or fused over time, rather than visible-only. But is it even feasible for a network to learn amodal? we dont know...

Two anomalies, explained. *Noisy depth beats exact depth* (0.77 vs. 0.72; 10 episodes won, 0 lost, $p = 0.002$), yet its map is worse (mIoU 0.277 vs. 0.288). All 56 exact-depth failures run out of steps while turning in place (at most 38 forward moves in 500 steps), the same failure every planner shows on visible labels: on a visible-only map each 10° turn changes which cells are known, the aim flips sides, and the controller's dither guard does not catch it. Depth noise breaks the cycle in some episodes. It is a controller artefact of single-frame maps. Fusion hurts noisy depth (0.77 to 0.69, 34 lost vs. 18 gained, $p = 0.037$), and 27 failures become "no local

		Closed loop, R1 planner (200 episodes)				Map quality vs. amodal R1		
Source	Fusion	Success [95% CI]	SPL	Coll.	Fail: budget / no plan	mIoU	free prec.	c
amodal labels (privileged)	–	.955 [.92, .98]	.944	.32	3 / 6	1	1	
2D LiDAR	–	.940 [.90, .97]	.929	.54	12 / 0	.351	.906	
	log-odds	.940 [.90, .97]	.930	.53	12 / 0			
3D LiDAR	–	.960 [.92, .98]	.949	.46	8 / 0	.409	.924	
visible depth, exact	–	.720 [.65, .78]	.672	.32	56 / 0	.288	.955	
	log-odds	.925 [.88, .95]	.904	.27	15 / 0			
visible depth, noisy	–	.770 [.71, .82]	.727	.30	46 / 0	.277	.958	
	log-odds	.690 [.62, .75]	.670	.21	35 / 27			
	symmetric	.770 [.71, .82]	.749	.24	28 / 18			
	log-odds, noisy odom.	.745 [.68, .80]	.718	.20	6 / 45			

Table 2: Perception sources. "Visible depth" back-projects the camera's own depth image (the input a monocular network would replace); "noisy" scales each image by $\mathcal{N}(1, 0.05)$ and adds 2% per-pixel noise. Log-odds fusion uses free -0.85 / occupied $+1.4$; "symmetric" uses ± 1.0. Exact visible depth reproduces the visible labels exactly. Map quality is for single frames, so fusion rows are blank.

plan". The cause is the asymmetric log-odds update: each mis-scaled frame adds a large occupied increment that later frames clear only slowly. With symmetric ± 1.0 updates the loss disappears (0.77, $p = 0.007$ against the asymmetric rule), while for exact depth the rule makes no difference (0.925 vs. 0.915). Odometry noise (2%, 1°) had no measurable effect ($p = 0.17$). We need to un further test to confirm that the blocked cells are doorways.

Collisions. Mean collisions per episode are driven by a few episodes in which the robot slides along a wall for its whole step budget: two oracle R5 episodes hold 99.6% of its collisions, and one 2D-LiDAR episode holds 58% of that source's. We therefore report the share of episodes with any collision. By that measure the 2D LiDAR's excess over the 3D one (54% vs. 46%) matches its blindness to overhangs.

5 Future Directions and Timeline

Stage 2 decision. Now that the minival noise study is done (Section 4). We are unsure how to proceed, we are stuck between whether we should try creating the learned representation for all representations or just a select few based on the results above.

Label fixes before training.

- Replace the 8-connected distance with fast marching (or any-angle paths for R4, R6 and R7, removing the 45° snapping in R7 and the length bias.
- Clip or log-scale the R6 (and R4) regression targets, sample several goals per frame for goal-conditioned heads; guard the unreachable-robot case in R4.
- Use the same occupancy threshold rule for visible and amodal labels.

Planner fixes before evaluating predictions.

- Choose the local R4 target among cells connected to the robot inside the crop (fixes R4 A* and R6+R4 in the closed loop).
- Make the controller's dither guard catch heading flips larger than 25° (the turn-in-place failures of visible-only maps).
- Use symmetric log-odds when fusing predicted maps, and check whether the cells it closed were doorways.

Stage 3: monocular predictor. Need to a lit review, but maybe a Lift-Splat-style network with indoor depth range, height-banded splatting and a pretrained backbone (ResNet-50 / DINOv2). Training data from more HM3D scenes, split by scene; MP3D for generalisation

Stage 4 and ablations. Odometry-based fusion of predictions; ablations: visible versus amodal supervision, single-frame versus accumulated.

Dates (2026)	Work
Nov 6–9	Planner fixes, label fixes (R7, target clipping) and Stage-2 decision
Nov 10–16	Generate training data, train baseline Lift-Splat on the chosen representation(s) and conduct extensive Lit Review for choosing our architecture
Nov 17–20	Create and train our chosen architecture, compare on navigability metrics and map-quality metrics
Nov 21–23	Closed-loop evaluation, accumulation (Stage 4), ablations, final report

Table 3: Timeline for the remainder of the project.